计算机操作系统

第3章 处理器管理（1）

程序与系统的工作流程

- 程序应具体两个基本特点
  - 顺序性
  - 可再现性
- 系统的工作流程分为顺序执行和并发执行
  - 顺序执行的工作方式具有两个特征
    - 封闭性——独占全部资源，不受其他程序影响
    - 可再现性 ——重复运行的结果相同
  - 并发执行的工作方式中，多道程序
    - 宏观上看，同时执行
    - 微观上看，轮流执行
  - 并发执行的工作方式具有三个特征
    - 随机性——运行时情景随机
    - 不可再现性——重复运行结果可能不同
    - 相互制约——资源争夺和协作关系

并发执行的不可再现性

- 例子:已知两道程序PA和PB，它们对同一个变量count进行操作，PA程序每次运行时对变量count进行加1操作，而PB程序每次运行时对变量count进行减1操作。我们用C语言语法描述PA和PB程序如下：

PA(){	PB(){
int x;	int y;
① x=count;	④ y=count;
② x=x+1;	⑤ y=y-1;
③ count=x;	⑥ count=y;
}	}

  - 在count=100时PA()和PB()各运行一次，那么，在并发执行方式下，它们运行后count的值是多少?
  - ①②③④⑤⑥执行： count=100
  - ①④⑤⑥②③执行： count=101
  - ④①②③⑤⑥执行： count=99

进程

- Dijkstra：一道程序在一个数据集上的一次执行过程，称为一个进程(Process)
- IBM：程序的运行过程称为任务（Task）
- 进程和任务都是运行着的程序的描述，二者可以互换使用，不加区别。
- 进程的主要特征
  - 动态性：具有生命周期，状态不断变化
  - 并发性：可以并发执行
  - 独立性：互相独立，只能访问自身的内存空间
  - 结构性：不同的进程可抽象出相同的属性，从而抽象结构相同
  - 异步性：运行时情况不可预知

进程的动态性

- 进程的三种基本状态
  - 就绪：能够运行但却未在被处理器处理的状态
  - 运行：正在被处理器处理的状态
  - 阻塞：由于各种原因（IO操作、资源缺乏等）暂时无法运行的状态

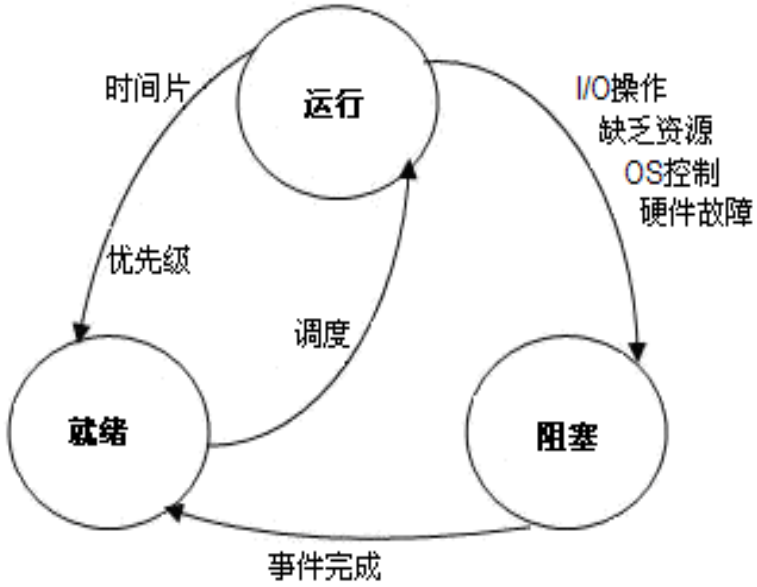
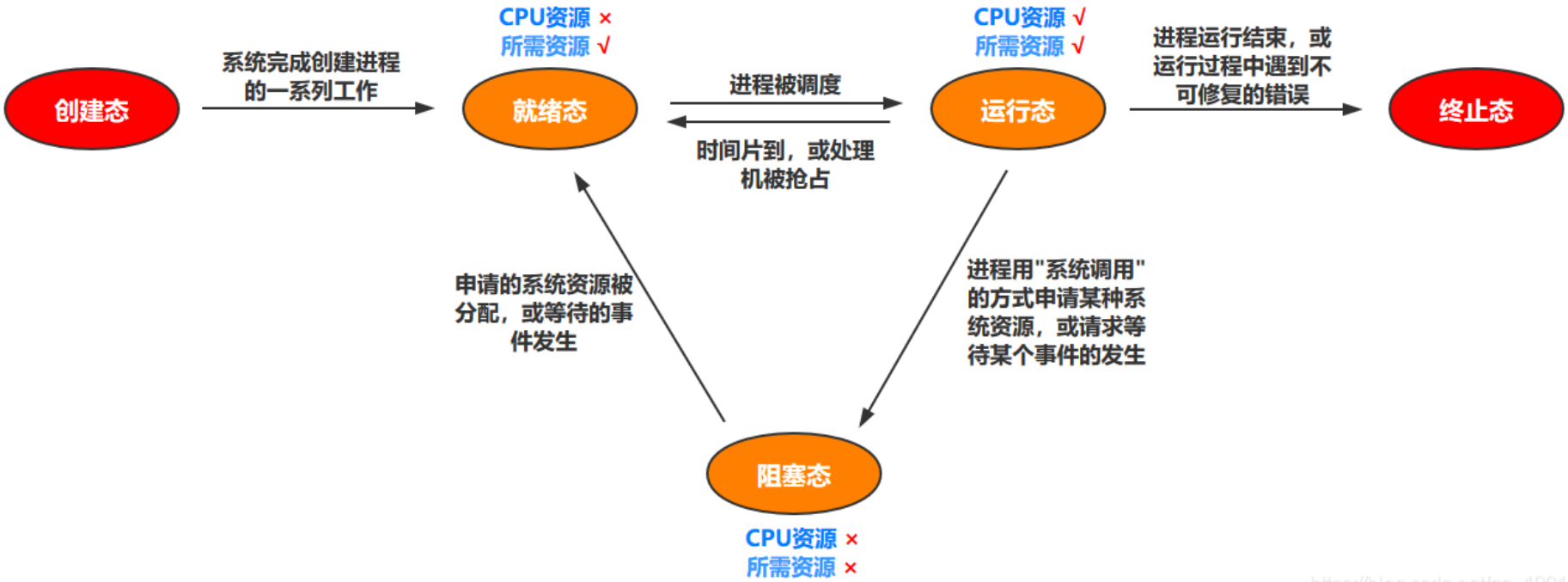


图3-1 进程状态转换关系图

五态(补充)



https://blog.csdn.net/qq_19018277

进程的另外两种状态

- 创建态 (New，又称：新建态)：进程正在被创建，操作系统为该进程分配系统资源、初始化PCB
- 终止态 (Terminated，又称：结束态)：进程正在被撤销，操作系统会回收该进程拥有的系统资源、撤销PCB

https://blog.csdn.net/qq_19018277

七态---挂起(补充)

- 进程的另一种状态：挂起
  - 为什么要进程挂起？为了合理且充分地利用系统资源
  - 什么是进程挂起呢？进程在挂起状态时，意味着进程没有占用内存空间，放在外存空间
  - 引起进程挂起的原因？
    - 终端用户的请求
    - 父进程的请求
    - 负荷调节的需要
    - 操作系统的需要
    - 对换的需要

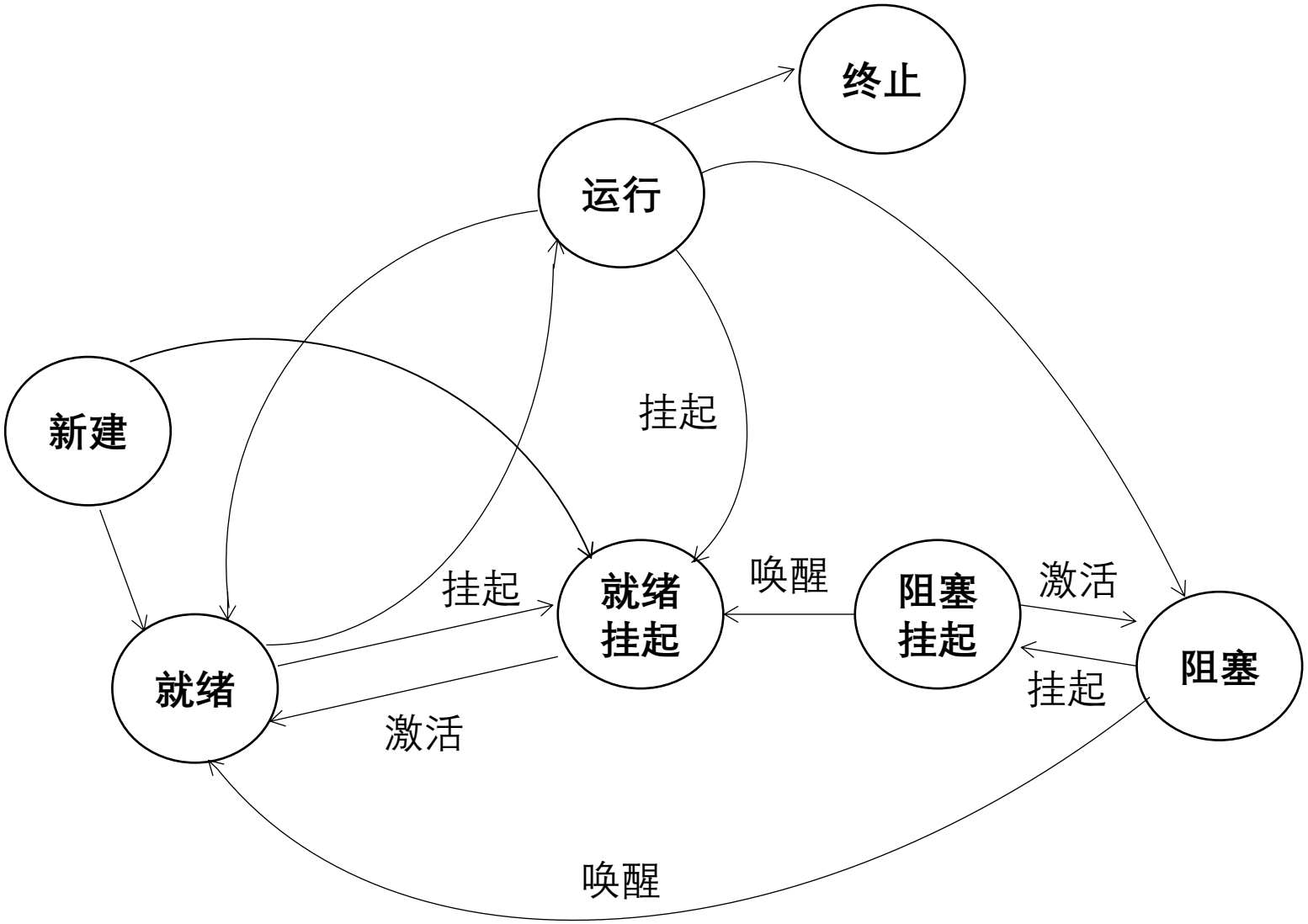
七态---与挂起相关的状态转换(补充)

- 挂起：把一个进程从内存转到外存
  - 阻塞到阻塞挂起：没有进程处于就绪状态或就绪进程要求更多内在资源时，会进行这种转换，以提交新进程或运行就绪进程
  - 就绪到就绪挂起：当有高优先级阻塞(系统认为会很快就绪的)进程和低优先就绪进程时，系统会选择挂起低优先级就绪进程
  - 运行到就绪挂起：对抢先式分时系统，当有高优先级阻塞挂起进程因事件出现而进程就绪挂起时，系统可能会把进程转到就绪挂起状态

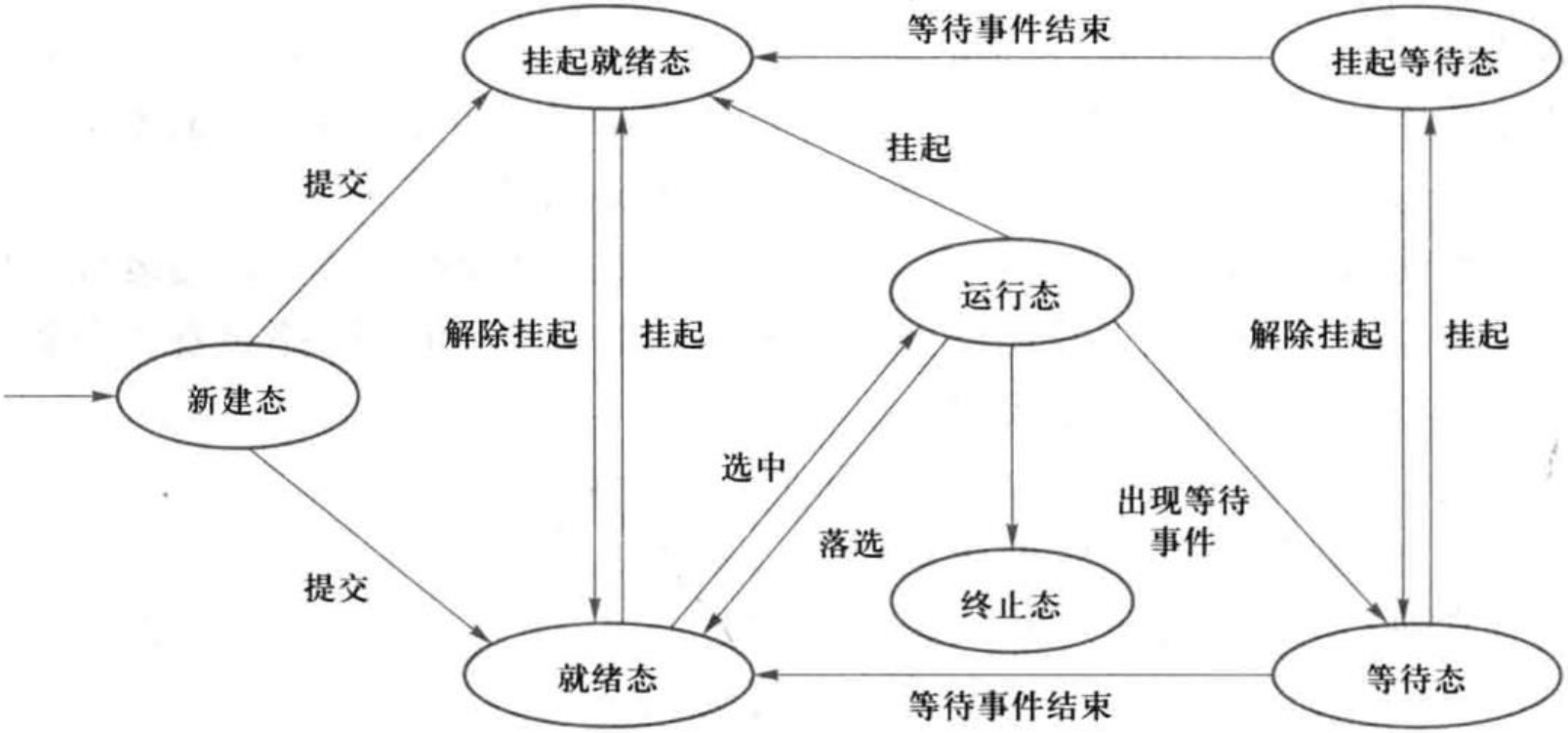
七态---与挂起相关的状态转换(补充)

- 解挂/激活：把一个进程从外存转到内存
  - 就绪挂起到就绪：没有就绪进程或挂起就绪进程优先级高于就绪进程时，会进行这种转换
  - 阻塞挂起到阻塞：当一个进程释放足够内存时，系统会把一个高优先级阻塞挂起(系统认为会很快出现所待的事件)进程转换为阻塞进程
- 唤醒：在外存时的状态转换
  - 阻塞挂起到就绪挂起：当有阻塞挂起进程因相关事件出现时，系统会把阻塞挂起进程转换为就绪挂起进程

七态---与挂起相关的状态转换(补充)



七态---与挂起相关的状态转换(补充)



具有挂起功能的系统的进程状态及其转换示意图

进程控制块（PCB）

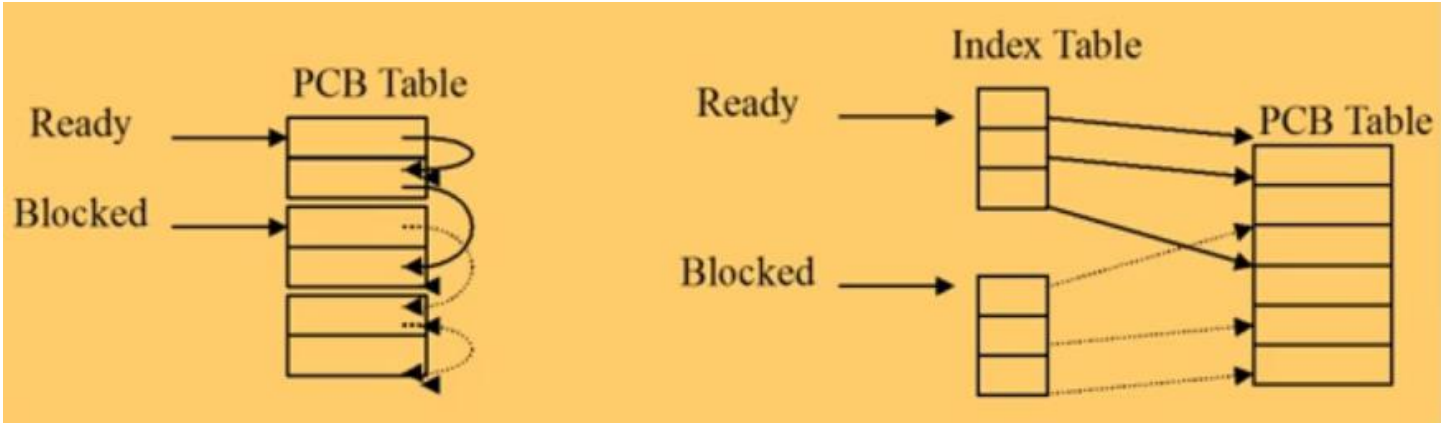
- 进程控制块（PCB）是操作系统用以描述进程信息的数据结构，由三部分组成：
  - 基本信息部分（至少包括）
    - 进程名：通常是程序名
    - 进程标识符(pid)：操作系统用以识别进程的唯一标识
    - 用户标识：创建进程的用户标识
    - 进程状态：运行、就绪或阻塞
  - 管理信息部分（至少包括）
    - 程序和数据地址：程序和数据在内存中的位置信息
    - IO操作相关参数：进程进行IO操作时的参数
    - 进程通信信息：用于进程间通信的消息队列指针等
  - 控制信息部分（至少包括）
    - 现场信息：进程离开运行状态时，保存进程现场以备恢复
    - 调度参数：与进程调度相关的参考信息，如优先级、运行时间等
    - 同步、互斥信息量：用户保证进程间同步、互斥关系的信息

PCB队列

- 一个进程唯一对应一个PCB，一个PCB也唯一对应一个进程。在多道程序设计环境中，可能有多个进程，操作系统用队列来管理这些进程，称为进程队列或PCB队列。PCB队列往往有多个，可以分为两类：
  - 就绪队列：就绪进程的PCB组成的队列，按照进程性质或优先级，系统中可以有多个就绪队列，以提高进程调度算法选择进程的效率。在多处理器系统中，就绪队列也被称作请求队列，每个处理器具有独立的请求队列。
  - 等待队列：阻塞进程的PCB组成的队列，按照阻塞原因，系统中可以有多个等待队列，以便在阻塞原因解除时统一调整进程状态。

PCB队列(补充)

- PCB队列的组织方式（存储）
  - 链接方式：把同一状态的PCB，用其中的链接字链接成一个队列。如：就绪队列、阻塞队列（根据不同阻塞原因）。
  - 索引方式：建立就绪索引表、阻塞索引表等。把索引表在内存的首地址放在内存的专用单元中。



进程管理

- 操作系统进程管理的主要功能是：
  - 控制：对进程生命期及其状态转换的实现；
  - 同步：对并发执行进程的控制，以保证程序的可再现性和任务协作；
  - 通信：实现进程之间的数据交换；
  - 调度：实现并发执行在微观上的轮流交替运行；
  - 死锁：分析、解决并发执行的另一种错误现象。

进程控制

- 改变进程状态的操作不应当被打断，即具有原子性（All or Nothing）。
- 原语（Primitive）是特殊的程序段，这样的程序段中的所有指令要么全部执行要么一个都不执行。即原语是具有原子性的程序段。
  - 原语也称广义指令，原语中的几条指令被看成一个实体。
  - 原语中的指令若执行不成功，系统需要回退到原语执行前的状态。
  - 原语的主要作用是保证系统运行的一致性。

进程控制-创建

- 进程控制就是实现对进程状态的转换，通过一组原语来实现，进程控制原语主要有创建、撤销、阻塞、唤醒和切换等。
- 1. 进程的创建原语（Create）用于创建一个进程
  - Create可能在以下情况下被使用：
    - 操作系统启动过程中,由初始化程序自动创建
    - 按照作业调度程序的作业步创建
    - 按照用户提交的命令
    - 用户程序使用了创建进程的系统调用（例如UNIX的fork）
  - Create被使用时完成以下操作：
    - 建立PCB
    - 生成进程标识符（pid）
    - 初始化进程和PCB
    - 将PCB加入适当的就绪队列

进程控制-控制树

- 作业调度程序运行时会创建进程，命令解释程序也会创建进程，而用户程序运行时（进程中）也可以通过系统调用创建进程，于是进程呈现树状结构（进程树），体现进程之间彼此创建的关系。

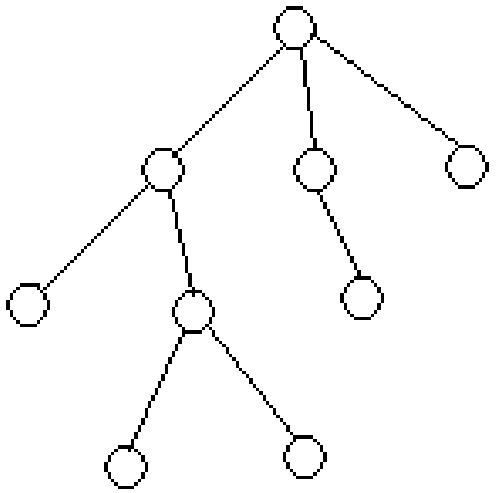


图3-2 进程树

进程控制-撤销

- 2. 进程的撤销原语（Destroy）用于结束一个进程
  - Destroy可能在以下情况下被使用：
    - 进程执行完毕
    - 进程出错而结束
    - 父进程结束
    - 人为终止
  - Destroy被使用时完成以下操作：
    - 进程资源回收
    - 删除进程PCB

进程控制-阻塞

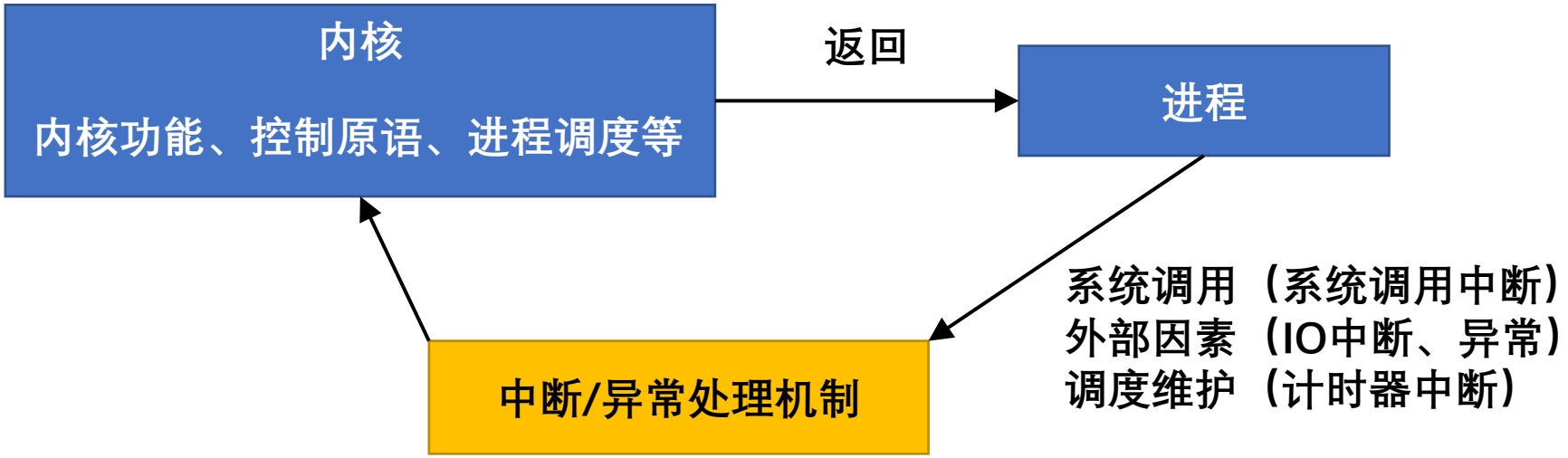
- 3. 进程的阻塞原语（Block）用于使一个进程进入阻塞状态
  - Block在阻塞原因发生时被使用。
  - Block被使用时完成以下操作：
    - 修改进程状态为阻塞状态
    - 将进程的现场保存，以备恢复
    - 将进程PCB加入合适的等待队列

进程控制-唤醒

- 4. 进程的唤醒原语（Wakeup）用于使一个进程从阻塞状态进入就绪状态
  - Wakeup在阻塞原因解除时被使用。
  - Wakeup被使用时完成以下操作：
    - 将进程PCB从等待队列中移除
    - 修改进程状态为就绪状态
    - 将进程PCB加入合适的就绪队列
  - 阻塞原因解除后，进程未必可以直接运行，因此合理的唤醒原语并不会将阻塞状态直接修改为运行状态。

进程控制-切换

- 5. 进程切换使处理器的控制权在进程之间转换，无法通过操作系统内核独立完成，必须借助硬件来实现。



相关进程间的制约关系-间接制约

- 例子：两个进程P1和P2分别使用同一台打印机逐行打印各自文本

P1程序	P2程序
…	…
打印第1行A1	打印第1行B1
打印第2行A2	打印第2行B2
…	…
打印第n行An	打印第m行Bm
…	…

第 1 行 A1
第 2 行 A2
第 1 行 B1
第 3 行 A3
…
第 m 行 Bm
第 n 行 An

图3-4 例1进程并发执行的一种结果

- 间接制约关系：仅由于使用相同的资源而造成的制约

相关进程间的制约关系-直接制约

- 例子：某任务需要通过循环使用read、move、write三个操作将源磁盘上的几个文件复制到目标磁盘。（一次操作处理一个文件）

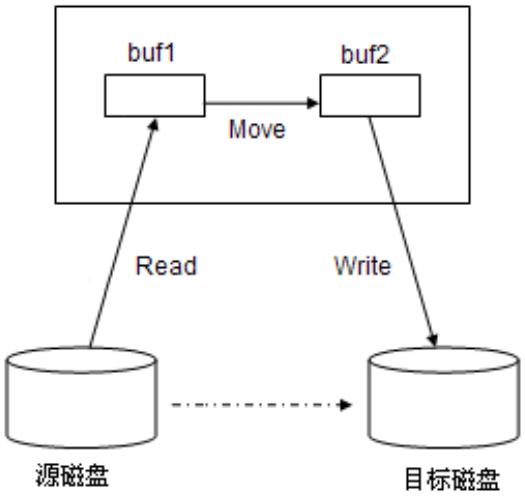


图3-5 三个进程的任务协作

- 若三操作在同一进程中顺序执行，则当进行IO操作（read和write）时，进程处于阻塞状态
- 若为三个操作分别建立进程，并发执行，则节约运行时间
- 然而，由于并发执行时三操作执行顺序的随机性，可能造成重复拷贝文件或遗漏文件的错误。

- 直接制约关系：由于进程间协作关系造成的制约

相关进程的两种制约关系

- 间接制约：进程间由于共享某种系统资源而形成的相互制约。



- 直接制约：进程间协作关系形成的制约。



临界资源与间接制约

- 资源的使用步骤：在操作系统的控制下，进程对系统资源的访问逻辑上需经历三个步骤：申请、使用、归还。
- 临界资源
  - 某些系统资源，在进程从申请到归还的周期中只能允许此进程独占，否则将造成运行错误，则称这样的资源为临界资源。
  - 常见的临界资源有：打印机、存储单元、文件 等。
- 间接制约
  - 间接制约实质上就是一组并发进程共享某种临界资源的制约关系。

临界区与互斥关系

- 临界区
  - 是指进程对应的程序中访问临界资源的一段程序代码，即进程从申请资源开始至归还资源为止的一段程序代码。
  - 相关临界区：这些临界区访问的是同一个临界资源
  - 无关临界区：不同临界资源之间的临界区
- 互斥关系
  - 对于一组（在某资源上）存在间接制约关系的并发进程，可以通过建立一种机制来避免制约关系导致的运行错误：当一个进程在这种临界资源对应的临界区内执行时，其他要求进入相关临界区的进程必须等待。
  - 若上述机制得以建立，则称这些并发进程为互斥进程，它们之间间建立了互斥关系。

直接制约与同步关系

- 对于一组存在直接制约关系的并发进程，为了避免制约关系导致的运行错误，应当使各进程中代码的运行顺序服从其内在要求，需要建立进程间单向或相互的依赖关系。
- 若一组进程中的每一个进程都与其他至少一个进程建立了单向或相互的依赖关系，则称这组进程为同步进程，这组进程间建立了同步关系。

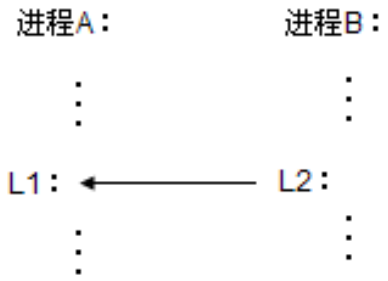


图3-8 单向依赖关系

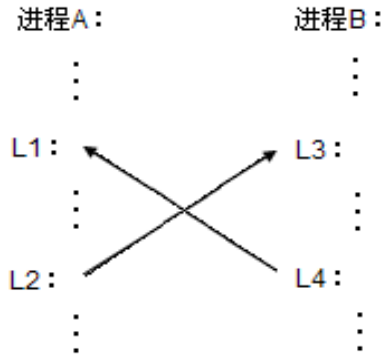


图3-9 相互依赖关系

进程同步机制

❖ 进程同步机制

- ➢ 用于控制并发进程的互斥、同步关系，保证它们能够正确执行的方法称为进程同步机制
- ➢ 常用的进程同步机制
  - 加锁机制（互斥）
  - 标志位机制（互斥）
  - 信号量机制（互斥/同步）
  - 管程机制（同步）

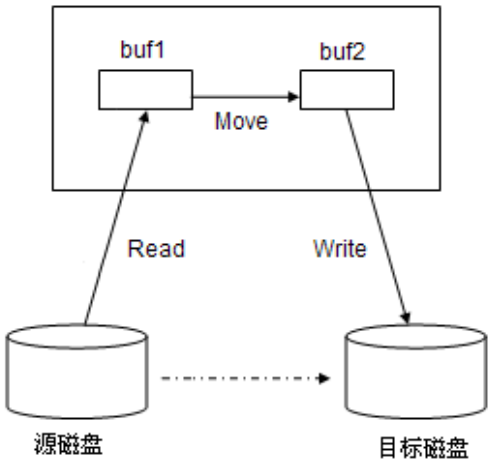


图3-5 三个进程的任务协作

临界区管理(同步机制遵循的原则)

❖ 实现互斥关系的关键是对临界区的执行进行有效控制。

临界区管理的准则有：

1）空闲让进：如果进程请求进入临界区，而没有其他进程在相关临界区内执行，则应当允许进入。

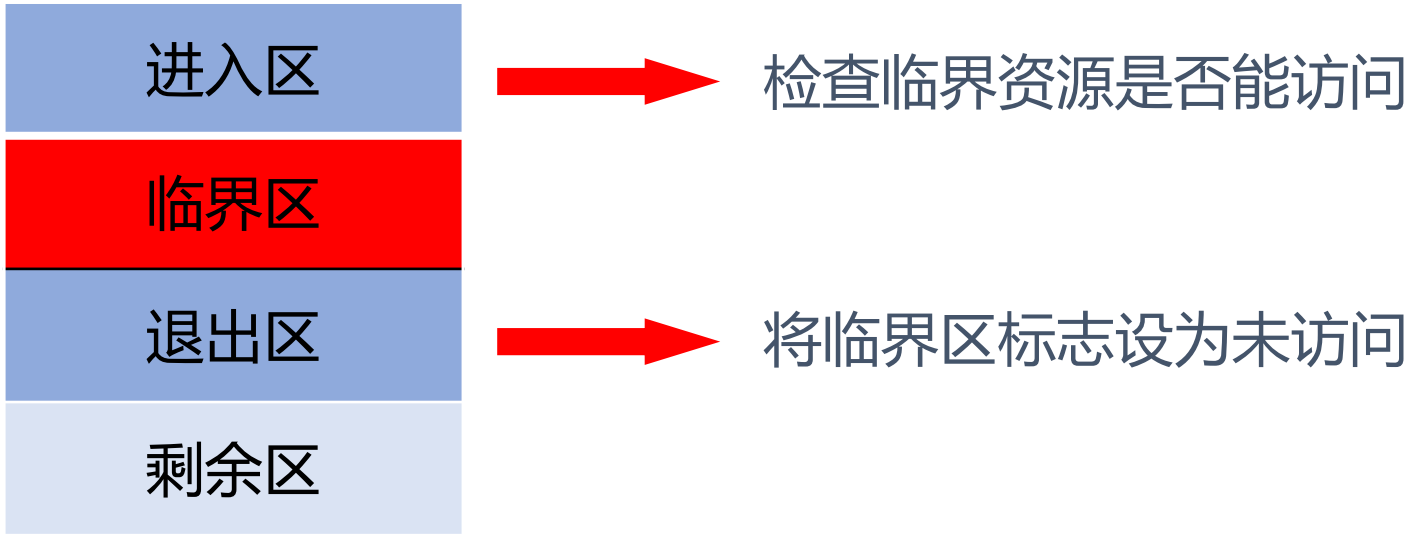
2）忙则等待：如果有进程在临界区内执行，则其他要求进入相关临界区的进程都需等待。

3）有限等待：对于要求进入临界区的进程，等待的时间应当是有限的。

4）让权等待：当一个进程离开临界区时，应当让请求进入相关临界区的下一个进程进入临界区执行。

访问临界区的循环进程描述

repeat



until false;

互斥关系的实现-加锁机制

- 加锁机制由三个要素组成：锁变量key、加锁操作lock(key)和解锁操作unlock(key)。

lock(key)
{
while(key==1);
key=1;
}
	unlock(key)
	{
	key=0;
	}

  - 这个机制中，锁变量key=1时表示上锁状态，反之则未上锁。
  - 当进程申请资源时需对资源加锁，若资源已上锁，则上锁操作将进行反复测试（等待），直到资源被解锁；当进程归还资源时需对资源解锁。

互斥关系的实现-加锁机制

❖ 例子(例3-1)

PA() {	PB() {
int x;	int y;
lock(key);	lock(key);
① x=count;	④ y=count;
② x=x+1;	⑤ y=y-1;
③ count=x;	⑥ count=y;
unlock(key);	unlock(key);
}	}

- 锁变量key初值：0
- 锁变量对资源（变量count）加锁，使得临界区①②③与④⑤⑥建立互斥关系。

互斥关系的实现-加锁机制

- 高级语言（软件）实现的加锁机制并不能实现真正的加锁，因为加锁操作内部仍可能被打断：

进程A的加锁	进程B的加锁
1： while(key==1);	3： while(key==1);
2： key=1;	4： key=1;

  - 当key=0时，若按1324执行，则两个进程同时进入临界区，加锁失效。

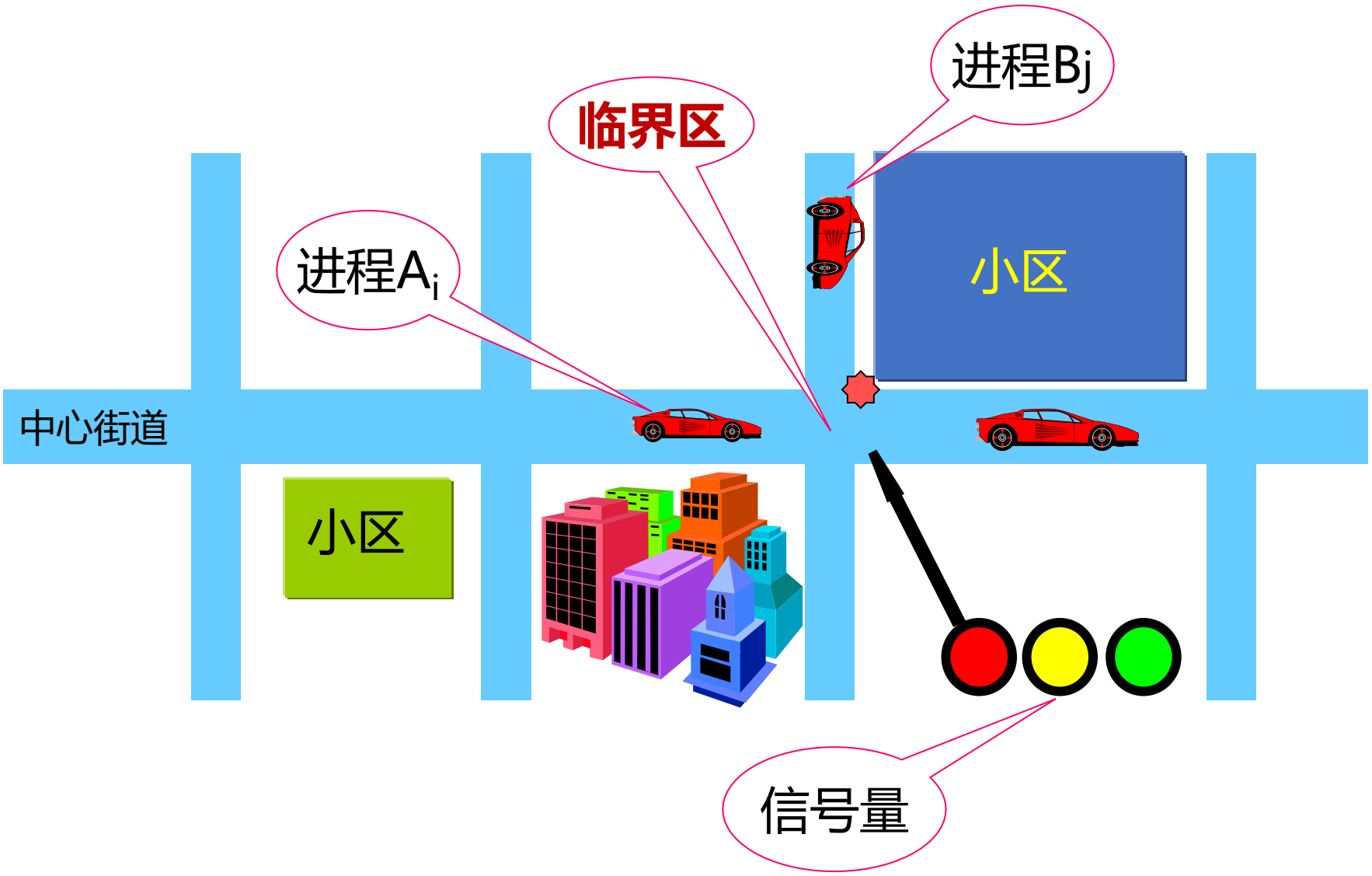
- 有效的加锁过程一般通过硬件指令完成，汇编代码如下：

```
tsl:
    mov ax, 1
    xchg ax, key     将key值保存到寄存器AX，并置key为1
    cmp ax, 0
    jne tsl          检查寄存器AX的值（原先的key值），不为0则循环
```

加锁机制总结

- 加锁机制的缺陷：
  - 普通的加锁机制不能实现互斥关系(保护临界区)，通常需要借助硬件支持(比如:原子操作指令);
  - 存在“忙等待”现象，浪费了处理器时间
  - 存在“饥饿”(Starvation)现象，进程等待时间可能无限长
  - 多个锁变量的加锁操作可能造成进程死锁

信号量机制



互斥关系的实现-信号量机制

❖ 信号量机制由三个要素组成：信号量s、原语p和原语v。

```
struct semaphore {
    int value;      信号量的值
    PCB *bq;        由于该信号量而堵塞的进程的PCB队列（等待队列）
}
```

p(s) {	v(s) {
s.value = s.value – 1 ;	s.value = s.value + 1 ;
if (s.value＜0) blocked(s) ;	if (s.value≤0) wakeup(s);
}	}

- s.value>0表示资源就绪，否则表示有进程在使用或请求使用资源。
- 进程请求资源时需使用原语p，若资源非空闲，则进入由于s而导致的等待队列，直至正在使用资源的进程归还资源时使用原语v将其唤醒。

互斥关系的实现-信号量机制

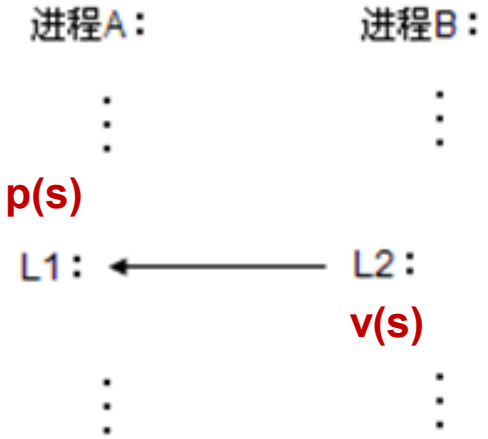
❖ 例子(例3-1)

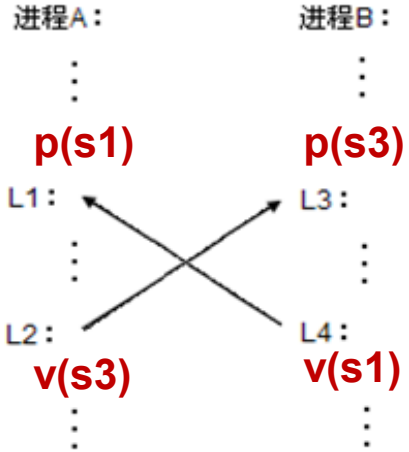
PA() {	PB() {
int x;	int y;
p(s);	p(s);
① x=count;	④ y=count;
② x=x+1;	⑤ y=y-1;
③ count=x;	⑥ count=y;
v(s);	v(s);
}	}

- 信号量s初值：1
- 用信号量s表示资源（变量count）的就绪情况，使得临界区①②③与④⑤⑥建立互斥关系。
- p和v皆为原语，不会出现如lock一般被打断的情况。

同步关系的实现-信号量机制

❖ 简单同步关系(单向依赖关系)/一般同步关系(相互依赖关系)





➢ 代码段L1对L2依赖时，可将L2的一次执行看作一种“资源”的归还，而L1的执行，需要申请使用这种“资源”。于是，可以类似地使用信号量机制实现同步关系：在L1执行前执行原语p，而在L2执行后执行原语v。与互斥关系不同的是，L1使用“资源”后并不归还（“资源”被消耗）。

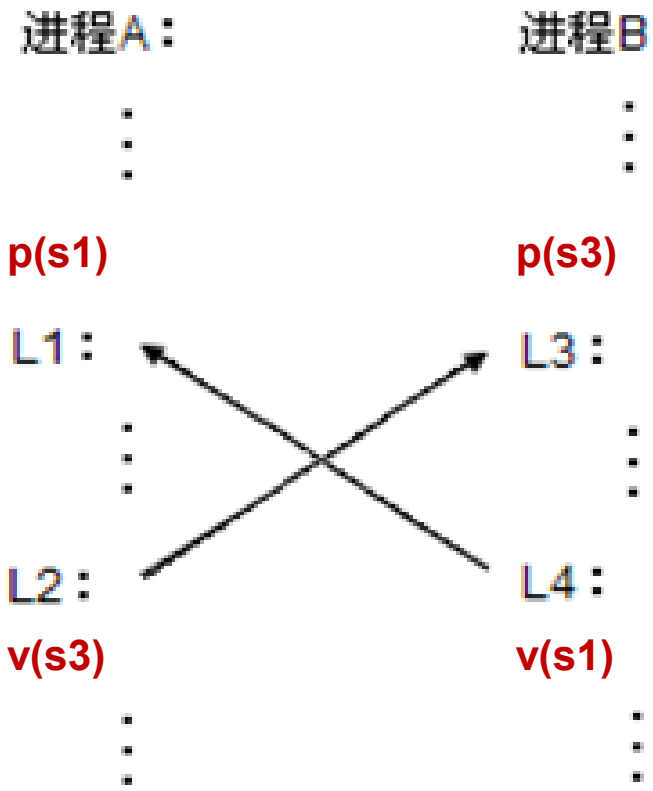
同步关系的实现-信号量机制

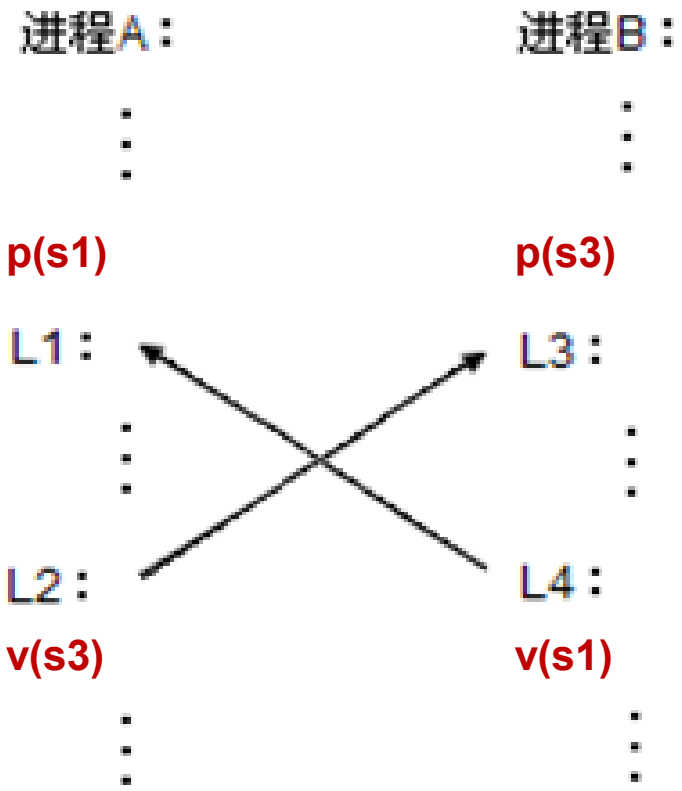
- 简单同步关系(单向依赖关系)



同步关系的实现-信号量机制

❖ 一般同步关系(相互依赖关系)





同步关系的实现-信号量机制

- 例子(例3-4:文件拷贝问题) 进程P1和P2的关系为一般同步关系

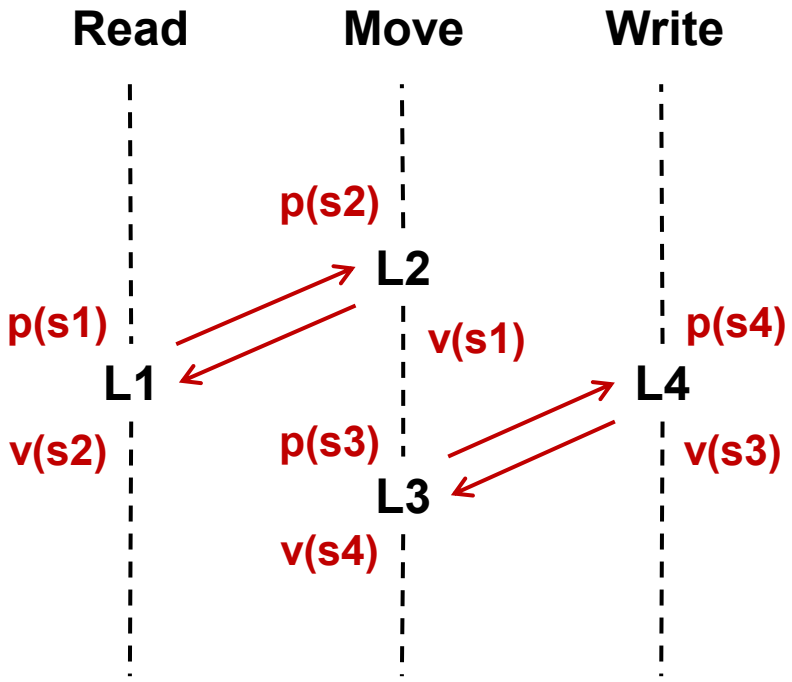
```
Semaphore s1=1,s2=0;
main(){
    cobegin{
        repeat P1();
        repeat P2();
        }
    }
```

```
P1() {
    计算并得到结果;
    p(s1);
    结果存入缓冲区 buf;
    v(s2);
}
```

```
P2() {
    p(s2);
    从缓冲区buf取出结果;
    v(s1);
    打印结果;
}
```

同步关系的实现-信号量机制

- 例子(例3-3:文件拷贝问题)
  - Read中的代码段
    - L1：将文件src的内容存入缓冲区buf1
  - Move中的代码段
    - L2：读出buf1中的内容
    - L3：将读出的内容写入buf2
  - Write中的代码段
    - L4：将buf2中的内容写入文件des

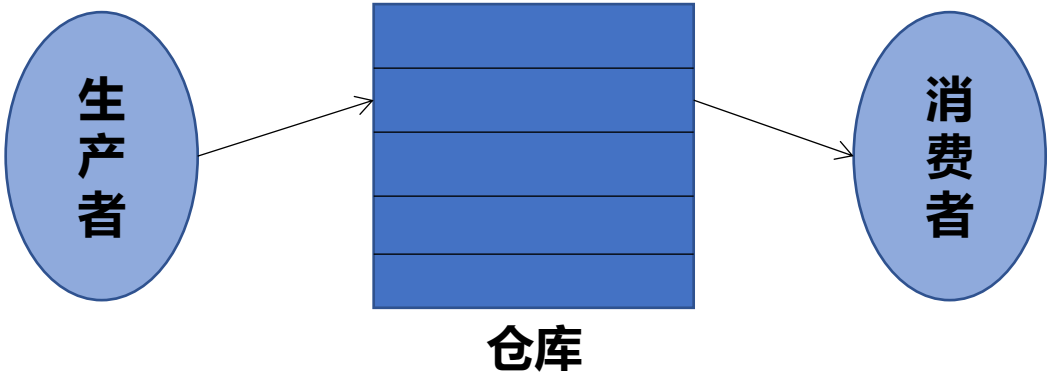


初始化：**s1=1; s2=0; s3=1; s4=0**

信号量机制总结

- 信号量机制相比于加锁机制的优势：
  - 实现进程的互斥关系
    - 采用原语，避免了加锁机制中被打断的情况。
    - 信号量机制把等待资源的进程置为阻塞状态，而非“忙等待”，提高了处理器的利用率。
    - 信号量机制中由操作系统原语wakeup唤醒进程，可采取一定策略（如先请求先服务等）而非如加锁机制中的“抢先者得”，这样可以避免“饥饿”现象发生。
  - 不仅可以实现进程的互斥关系，还可以实现进程的同步关系
- 信号量机制的缺点：
  - 不能够处理死锁问题;
  - 编程复杂度高;

生产者/消费者问题



- 用进程Pro和进程Con分别表示生产者和消费者，用缓冲区buf表示仓库，任务描述如下：

```
Pro() {
        生产一个物品;
        物品存入buf;
}
Con() {
        从buf取出物品;
        消费;
}
```

```
main(){
        cobegin{
                repeat Pro();
                repeat Con();
                }
        }
```

  - 存在问题?

生产者/消费者问题(简单PC问题)

- 生产者进程Pro的个数n=1，消费者进程Con的个数m=1，缓冲区buf单元格的个数k=1
  - 进程Pro和Con之间具有一般同步关系；
  - 初始化：Semaphore empty=1，full=0；
  - 存在问题?

Pro() { **生产一个物品;** **p(empty);** **物品存入缓冲区buf;** **v(full);** **}**	**Con() {** **p(full);** **从缓冲区buf取出物品;** **v(empty);** **消费;** **}**

生产者/消费者问题(一般PC问题1)

- 生产者进程Pro的个数n=1，消费者进程Con的个数m=1，缓冲区buf单元格的个数k>1
  - 进程Pro和Con之间不仅具有一般同步关系，还具有互斥关系；
  - 初始化：Semaphore mutex=1，empty=k，full=0；
  - 存在问题?
    - 连续两个或多个P操作时的顺序要求
    - 缺乏并行

```
Pro() {
        生产一个物品;
        p(empty);
        p(mutex);
        物品存入缓冲区buf[]的某个单元格;
        v(mutex);
        v(full);  }
```

```
Con() {
        p(full);
        p(mutex);
        从缓冲区buf[]的某个单元格取物品;
        v(mutex);
        v(empty);
        消费;  }
```

生产者/消费者问题(一般PC问题2)

- 生产者进程Pro的个数n=1，消费者进程Con的个数m=1，缓冲区buf单元格的个数k>1
  - 进程Pro和Con之间仅具有一般同步关系，不考虑互斥关系；
  - 存取方式：对每个缓冲区单元格buf[]设置空或满标记；
  - 初始化：Semaphore empty=k，full=0；

```
Pro() {
        生产一个物品;
        p(empty);
        找一个空标记的缓冲区单元格buf[x];
        物品存入 buf[x];
        设置buf[x]为满标记;
        v(full);
}
```

```
Con() {
        p(full);
        找一个满标记的缓冲区单元格buf[y];
        从buf[y]取物品;
        设置buf[y]为空标记;
        v(empty);
        消费;
}
```

生产者/消费者问题(一般PC问题3)

- 生产者进程Pro的个数n=1，消费者进程Con的个数m=1，缓冲区buf单元格的个数k>1
  - 进程Pro和Con之间仅具有一般同步关系，不考虑互斥关系；
  - 存取方式：按照先进先出方式循环存/取物品；
  - 初始化：Semaphore empty=k，full=0；int in=0，out=0；

```
Pro() {
        生产一个物品;
        p(empty);
        物品存入 buf[in];
        in=(in+1)%k;
        v(full);
}
```

```
Con() {
        p(full);
        从buf[out]取物品;
        out=(out+1)%k;
        v(empty);
        消费;
}
```

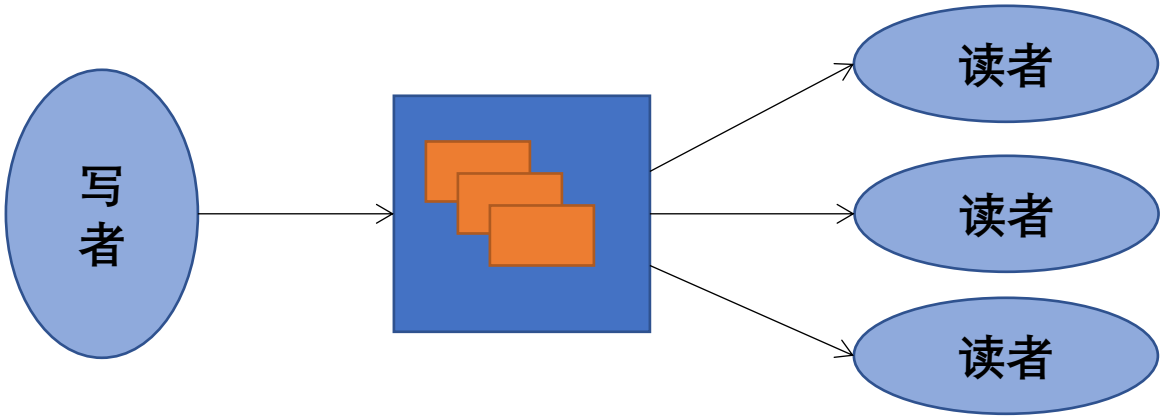
生产者/消费者问题(复杂PC问题)

- 生产者进程Pro的个数n>1，消费者进程Con的个数m>1，缓冲区buf单元格的个数k>1
  - 进程Pro和Con之间具有一般同步关系;
  - 进程Pro间具有互斥关系，进程Con间具有互斥关系。
  - 初始化：Semaphore empty=k,full=0,mutex1=1,mutex2=1;
    int in=0，out=0;

对于每一个生产者进程Pi, i=1,2,....n	对于每一个消费者进程Ci, i=1,2,....n

Pi() {
生产一个物品;
p(empty);
p(mutex1);
所生产的一个物品存入 buf[in];
in=(in+1)%k;
v(mutex1);
v(full);}
	Ci() {
	p(full);
	p(mutex2);
	从buf[out]取一个物品;
	out=(out+1)%k;
	v(mutex2);
	v(empty);
	消费;}

读者/写者问题



- 用一个进程Writer表示写者，用多个进程Reader表示读者，Writer和Reader的代码为

Writer() {
写操作;
}
	Reader() {
	读操作;
	}

  - 规定：①Writer进程与任一Reader进程之间互斥执行；
    ②多个Reader进程之间可以同时执行。
    如何实现?

读者/写者问题-1

- 用一个进程Writer表示写者，用多个进程Reader表示读者
  - 初始化：Semaphore ws=1;
  - Writer进程与Reader进程之间互斥执行，Reader进程之间也互斥执行
  - 存在问题?

Writer() {
p(ws);
写操作;
v(ws);
}
	对于任一Reader进程:
	Reader() {
	p(ws);
	读操作;
	v(ws);
	}

读者/写者问题-2

- 用一个进程Writer表示写者，用多个进程Reader表示读者
  - 初始化：Semaphore ws=1; int readers=0;
  - Writer进程与第一个和最后一个Reader进程之间互斥执行
  - 存在问题?

Writer() {
p(ws);
写操作;
v(ws);
}
	对于任一Reader进程:
	Reader() {
	readers=readers+1;
	If(readers==1) p(ws);
	读操作;
	readers=readers-1;
	If(readers==0)
	v(ws);
	}

读者/写者问题-3

- 用一个进程Writer表示写者，用多个进程Reader表示读者
  - 初始化：Semaphore mutex=1,ws=1; int readers=0;
  - Writer进程与第一个和最后一个Reader进程之间互斥执行；辅助变量的+1-1操作互斥执行。
  - 存在问题?

```
Writer() {
        p(ws);
        写操作;
        v(ws);
}
```

对于任一Reader进程:

```
Reader() {
        p(mutex);
        readers=readers+1;
        If(readers==1) p(ws);
        v(mutex);
        读操作;
        p(mutex);
        readers=readers-1;
        If(readers==0) v(ws);
        v(mutex);
}
```

读者/写者问题-4

- 用一个进程Writer表示写者，用多个进程Reader表示读者
  - 初始化:Semaphore mutex=1,ws=1,mutex0=1;int readers=0;
- 当写者进程要执行写操作而被阻塞时，正在执行的读操作可以继续，但新到来的读操作要等待---解决饥饿现象。

```
Writer() {
        p(mutex0);
        p(ws);
        写操作;
        v(ws);
        v(mutex0);
}
```

```
对于任一Reader进程:
Reader() {
        p(mutex0);
//虚拟一个申请操作;
        v(mutex0);
        p(mutex);
        readers=readers+1;
        If(readers==1) p(ws);
        v(mutex);
        读操作;
        p(mutex);
        readers=readers-1;
        If(readers==0) v(ws);
        v(mutex);
}
```

标志位机制/管程机制

- 标志位机制(对比锁机制)
  - 严格轮转互斥算法
  - Dekker互斥算法
  - Peterson互斥算法

- 管程机制(对比信号量机制)
  - 高级的同步机制
  - 面向对象的程序设计思想

进程同步总结

- 控制并发进程的互斥、同步关系，保证他们能够正确执行，称作进程同步。实现进程同步的方法称作进程同步机制。
- 操作系统需要为程序提供进程同步机制，而当需要处理互斥/同步关系时才决定如何使用这些机制。
  - 用于实现互斥关系（针对间接制约）的进程同步机制：加锁机制、信号量机制
  - 用于实现同步关系（针对直接制约）的进程同步机制：信号量机制
- 除加锁机制与信号量机制之外，常用的进程同步机制还有：标志位机制、管程机制 等。

进程同步总结

- 实现互斥关系常用方法及特点：
  - 硬件方法---禁用中断(仅限于单处理器)；
  - 软件方法---Dekker互斥算法/Peterson互斥算法等(复杂)；
  - 借助硬件支持---原子操作指令(单处理器或多处理器均可)；
- 实现同步关系常用方法及特点：
  - 信号量机制（经典/应用广泛）；
  - 管程（高级同步机制，体现面向对象设计的思想）；

同步-作业1

1. 某反复运行的业务流程由 4 道并发执行的工序 A、B、C、D 组成，其依赖关系描述如下：A、B 的第一次执行不受制约，而之后仅当执行一次 C 后方可执行一次 A，执行一次 D 后方可执行一次 B；A 与 B 各执行一次后方可执行一次 C 与一次 D。试在下列程序中填空，利用信号量机制，实现这一业务流程的同步关系。（每空可填入 0、1 或多条语句）

信号量初始化：_____

```
Pa()
{
    _____
    工序 A;
    _____
}
```

```
Pb()
{
    _____
    工序 B;
    _____
}
```

```
Pc()
{
    _____
    工序 C;
    _____
}
```

```
Pd()
{
    _____
    工序 D;
    _____
}
```

同步-作业2

2.虚拟打印机多任务并发控制。已知打印服务由一个打印队列 pntList、一个接收进程 Receive 和一个打印进程 Printer 组成。打印队列的数据结构以链表实现，用于保存用户提 交的打印请求，规定队列 pntList 最多可同时保存 100 个请求，pntList 的初始值为空；接 收 Receive 进程负责接收用户的打印请求，并将请求加入打印队列；打印进程 Printer 负责 从打印队列中移出打印请求，并将用数据送打印机打印。采用信号量机制，阅读下列 Receive、Printer 进程的并发程序设计，在空格中填入适当的信号量定义及其初值勤或者 p、 v 操作。

Semaphore _____，_____，_____；

Receive() {
接收用户的打印请求；
_____；
_____；
打印请求加入队列 pntList;
_____；
_____；
}

Printer() {
_____；
_____；
从队列 pntList 中移出一个打印请求；
_____；
_____；
打印。
}

同步-作业3

3.在一个图书馆的系统上运行着很多进程，其中一些进程查找图书信息称为读者，另一些进程更新图书信息称为写者，读者可以并发执行读操作，但是读操作和写操作不能并发执行，不同写者的写操作也不能并发执行。下面是可以满足上述要求的读者和写者程序，请填空使得程序完整正确。

```
Int   readcount;
Semaphore x=1,wsem=1;
Void reader()
{
    While(true)
    {
     P(x);
     (1)_____;
     If(readcount==1)
         (2)_____;
     V(x)
     READUNIT();
     P(x);
     (3)_____;
     If(4)_____)
         ((5)_____;
     V(x);
        }
}
Void writer()
{
    While(true)
    {
        P(wsem);
        WRITEUNIT();
        V(wsem);
    }
}
Void main()
{
    Readcount=0;
    Parbegin(reader,writer);
}
```

同步-测试题

有A、B两人通过信箱进行辩论赛，每个人都从自己的信箱中取得对方的问题。将答案和向对方提出的新问题组成一个邮件放入对方的邮箱中。假设A的信箱最多存放M个邮件，B的信箱最多存放N个邮件。初始时，A的信箱中有x个邮件(0<x<M), B的信箱中有y个邮件(0<y<N)。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下所示：

```
cobegin {
    process A
   {
     while(TRUE)
     {
       从A的信箱中取出一个邮件;
       回答问题并提出一个新问题;
       将新邮件放入B的信箱;
    }
     }
```

```
 process B
{
 while（TRUE）
  {
    从B的信箱中取出一个邮件;
    回答问题并提出一个新问题;
    将新邮件放入A的信箱;
  }
 }
} coend
```

当信箱不为空时，辩论者才能从信箱中取邮件，否则等待；当信箱不满时，辩论者才能将邮件放入信箱，否则等待。请添加必要的信号量和P、V操作，以实现上述过程的同步。要求写出完整的PV并发控制伪代码，并说明信号量的含义和初值。

计算机操作系统

第3章 处理器管理（2）

进程通信

- 进程通信
  - 两个或多个进程之间交换数据的过程，其中提供数据的一方称为发送进程，得到数据的一方称为接收进程
- 为什么需要进程通信?
  - 进程的独立性：不同进程无法互相访问数据和代码
  - 任务协作：进程相互协作通常都需要交换数据
- 进程通信提供2个操作:
  - send(message)
  - receive (message)
- 进程通信流程：
  - 在它们之间建立通信链路
  - 通过send/receive交换消息

进程通信

- 进程通信类型（后）
  - 低级通信/高级通信
  - 直接通信/间接通信
  - 通信链路
  - 数据格式
  - 收发操作的同步方式
- 进程通信方式（先）
  - 共享存储区通信
  - 消息缓冲通信
  - 信箱通信
  - 管道通信

进程通信的类型

- 低级通信：操作系统内核程序之间的通信，只能传递状态和整数值（控制信息），交换的数据量较小，且交换的数据用于控制进程的执行。
- 高级通信：应用程序之间的通信，交换的数据量可以很大，且交换的数据是接收进程的处理对象。

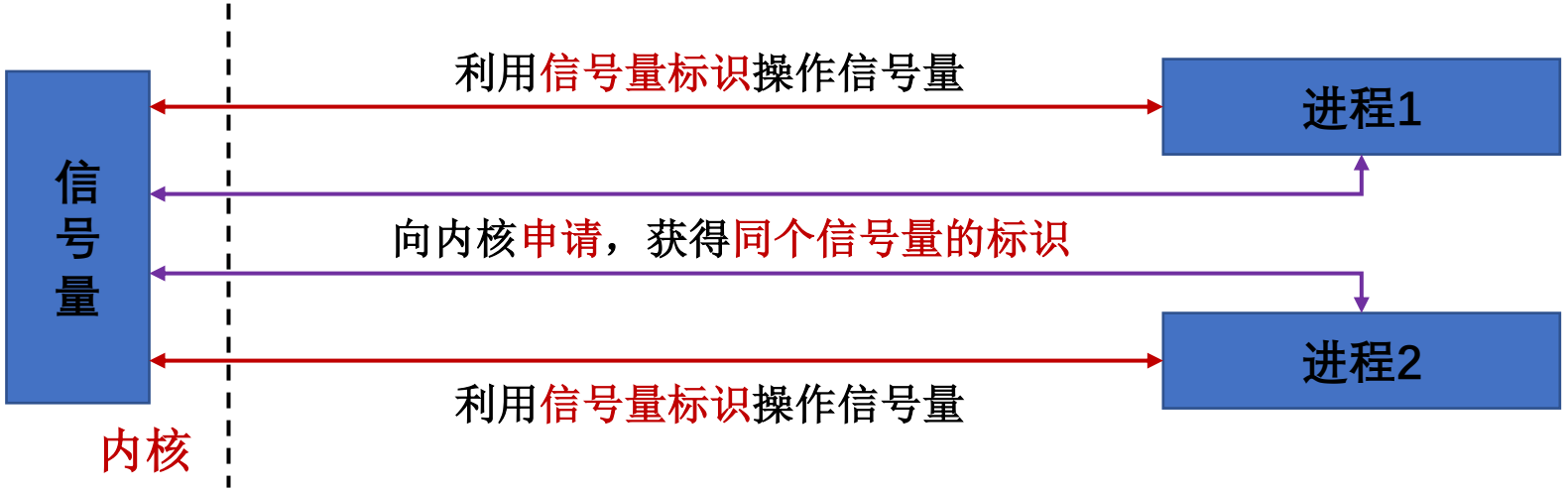
- 间接通信：借助于收发双方进程之外的共享区作为通信中转。通常收方和发方的数目可以是任意的。
- 直接通信：信息直接传递给接收方。

进程通信的类型

- 通信链路
  - 缓冲区的容量—队列的消息被附加到链路的方式
    - 0容量—发送方必须等待接收方
    - 有限容量—发送方在队列满的时候要等待
    - 无限容量—理想状况，不用等
  - 单向/双向
  - 点对点/多点/广播
- 数据格式
  - 字节流(byte stream)
  - 报文(datagram/message)
- 收发操作的同步方式
  - 阻塞被认为是同步的
  - 非阻塞被认为是异步的

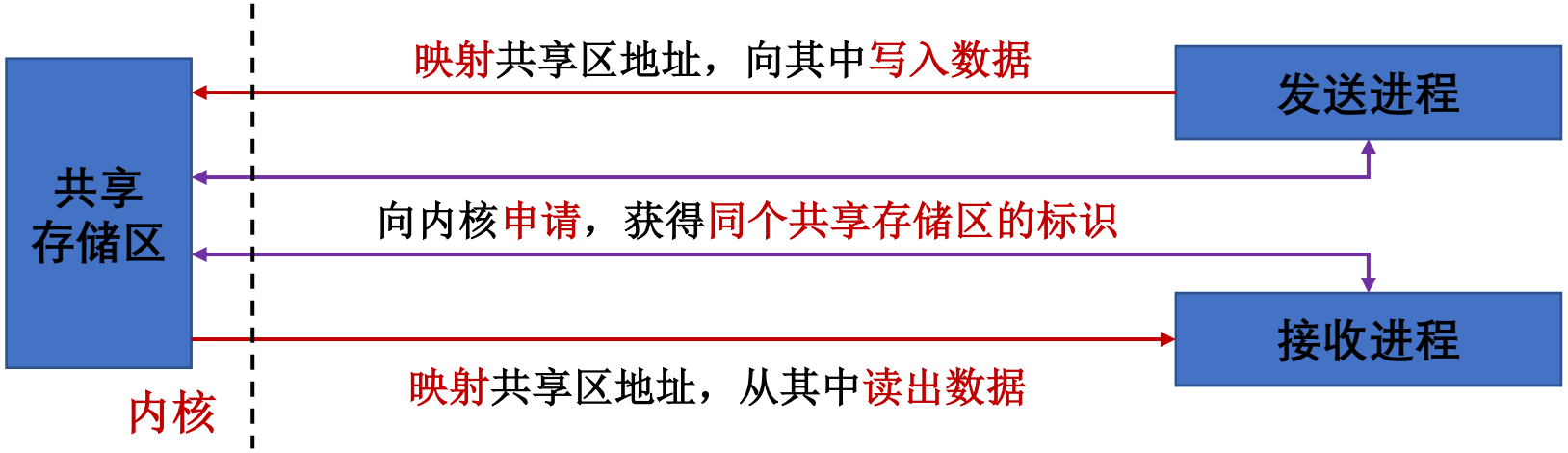
从信号量到进程通信

- 信号量是一种常见的共享数据——低级通信
- 信号量：数据结构的存放位置并非用户进程的私有空间，而是内核空间，信号量使用前需要向内核申请，以获得信号量标识。



共享存储区通信

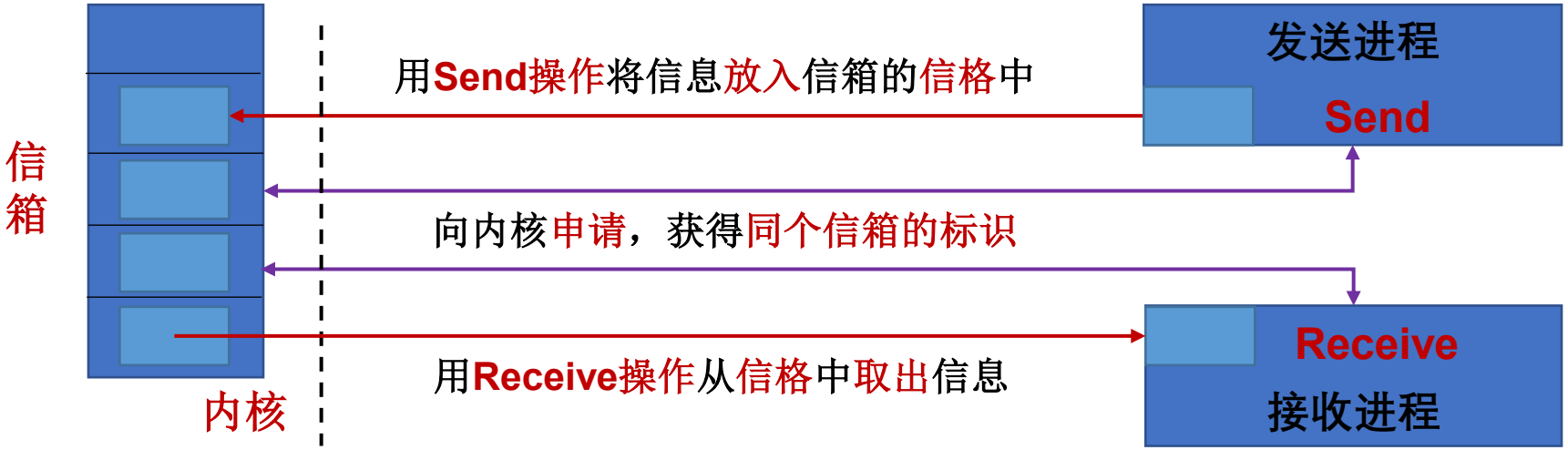
- 共享存储区通信：进程根据需要向内核申请位于内核空间的共享存储区，并将此区域映射到发送进程与接收进程的地址空间，使得发送/接收进程可以通过共享的存储区进行通信。



  - 共享存储区中的数据存取无固定模式，需注意根据实际情况处理同步/互斥关系

信箱通信

- 信箱通信：在内核空间中建立专门区域作为信箱，信箱中划分有限个信格，每个信格可存放一条信息（信件）。每个信箱可由多个发送进程与接收进程共享，发送进程用send操作将信息放入信格中，接收进程用receive操作从信格中取出信息。



  - 与共享存储区通信比较，信箱通信的数据存取模式固定，其实现是一个多生产者多消费者的PC问题。

信箱通信-信箱结构

- 信箱数据结构包括信箱头和信箱体。
  - 信箱头包含信箱的描述、控制信息：
    - 信箱名：信箱的名称
    - 信箱标识符ID：生成信箱时生成的唯一的标识符
    - 信箱大小size：信格总数
    - 同步信号量mailnum：表征信件数量的同步信号量
    - 同步信号量freenum：表征空信格数量的同步信号量
    - 读互斥信号量rnutex：保证多个接收者进程互斥的信号量
    - 写互斥信号量wmutex：保证多个发送者进程互斥的信号量
    - 读指针out：当前可读信件所在信格地址
    - 写指针in：当前可写入的空信格地址
  - 信箱体是由若干个连续的称为信格的区域组成的缓冲区数组。

信箱通信-信箱的实现

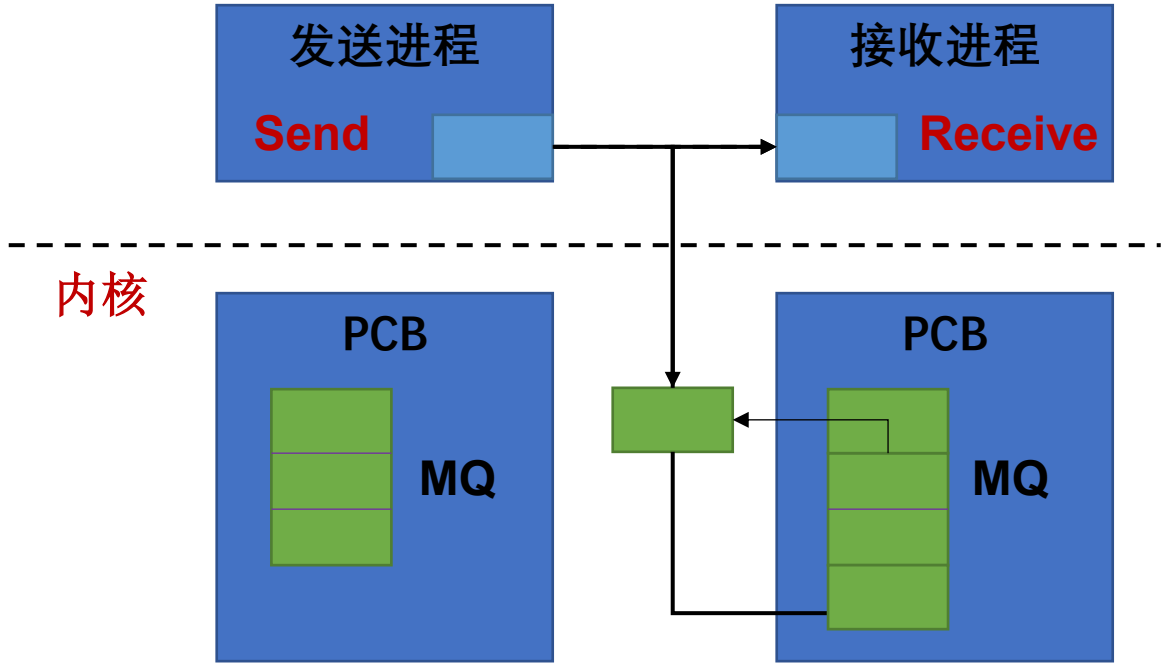
- 信箱通信中Send和Receive的实现（发送和接收操作）：

```
Send(toBoxID, & mailPtr) {
    根据信箱标识符toBoxID查找信箱toBox;
    p(toBox.freeNum);
    p(toBox.wMutex);
    toBox.buf [toBox.in] ← mailPtr;
    toBox.in = (toBox.in+1) % toBox.size;
    v(toBox.wMutex);
    v(toBox.mailNum);
}
```

```
Receive(fromBoxID, & mailPtr) {
    根据信箱标识符fromBoxID查找信箱fromBox;
    p(fromBox.mailNum);
    p(fromBox.rMutex);
    mailPtr ← fromBox.buf [fromBox.out];
    fromBox.out = (fromBox.out+1) % toBox.size;
    v(fromBox.rMutex);
    v(fromBox.freeNum);
}
```

消息缓冲通信

- 消息缓冲通信：内核空间中为每个进程配备一个消息队列MQ，并记录在进程的PCB中。发送进程通过Send操作将信息封入一个消息缓冲区中，并追加到接收进程的消息队列内；接收进程通过Receive操作收取自身消息队列中的信息。



消息缓冲通信-数据结构

- 消息缓冲通信中，数据结构主要有消息缓冲区的结构 、PCB的通信参数结构。
  - 消息缓冲区的结构
    - 发送进程标识(pid)
    - 正文大小(size)
    - 正文(data)
    - 向下指针(next)
  - PCB的通信参数结构
    - 消息缓冲区队列(mq)
    - 互斥信号量(mutex)
    - 同步信号量(msg)

消息缓冲通信-实现

❖ 消息缓冲通信中Send和Receive的实现（发送和接收操作）：

```
Send(des, msgOut) {
    查找进程des的PCB：desPCB;
    申请一个空的消息缓冲区buf;
    将消息msgOut封入buf中;
    p(desPCB.mutex);
    将buf追加到desPCB.MQ;
    v(desPCB.mutex);
    v(desPCB.msg);
}
```

```
Receive(msgOut) {
    查找当前进程this的PCB：thisPCB;
    p(thisPCB.msg);
    p(thisPCB.mutex);
    从thisPCB.MQ中移出一个消息缓冲区buf;
    v(thisPCB.mutex);
    从buf中读出信息到msgOut;
    归还消息缓冲区buf;
}
```

➢ 与信箱通信相比，消息缓冲通信中的消息队列专属于进程。消息缓冲通信是一种直接通信，当接收进程未开启时，通信不成功。

管道通信

- 管道通信：通信双方共享一个文件，发送进程向文件中写入数据，接收进程从中读出数据，从而实现通信。共享的文件被作为“管道”。
  - 管道通信中的共享数据并不存放在内核空间中，而是存放在管道（文件）中，需注意维护数据的安全问题。
  - 管道通信中的管道（文件），同共享存储区类似，并无固定数据存取模式，因此需注意根据实际情况处理互斥/同步关系
  - 管道通信可以使用远程文件作为管道，因此，此种通信方式不局限于单计算机系统。

线程

❖ 例：编写一个MP3播放软件，核心功能模块有三个：

1. 从MP3音频文件中读取数据；
2. 对数据进行解压缩；
3. 把解压后的音频数据播放出来；

单进程的实现方法

```
main( )
{
    while(TRUE)
     {
I/O ———— Read( );
CPU ———— Decompress( );
        Play( );
     }
}
Read( ) { … }
Decompress( ) { … }
Play( ) { … }
```

问题：

- 播放出来的声音能否连贯？
- 各个函数之间不是并发执行，影响资源的使用效率；

怎么解决？ 利用进程可以并发执行的特点，分为三个进程执行。

线程

多进程的实现方法

```
程序1
main( )
{
   while(TRUE)
   {
      Read( );
   }
}
Read( ) { … }
```

```
程序2
main( )
{
   while(TRUE)
   {
      Decompress( );
   }
}
Decompress( ) { … }
```

```
程序3
main( )
{
   while(TRUE)
   {
      Play( );
   }
}
Play( ) { … }
```

问题：

- 进程之间如何通信，共享数据？
- 维护进程的系统开销较大：
  - 创建进程时，分配资源，建立PCB
  - 撤销进程时，回收资源，撤销PCB
  - 进程切换时，保存当前进程的状态信息

怎么解决？需要提出一种新的实体，满足以下特征：

实体之间可以并发地执行；

实体之间共享相同的地址空间。

线程

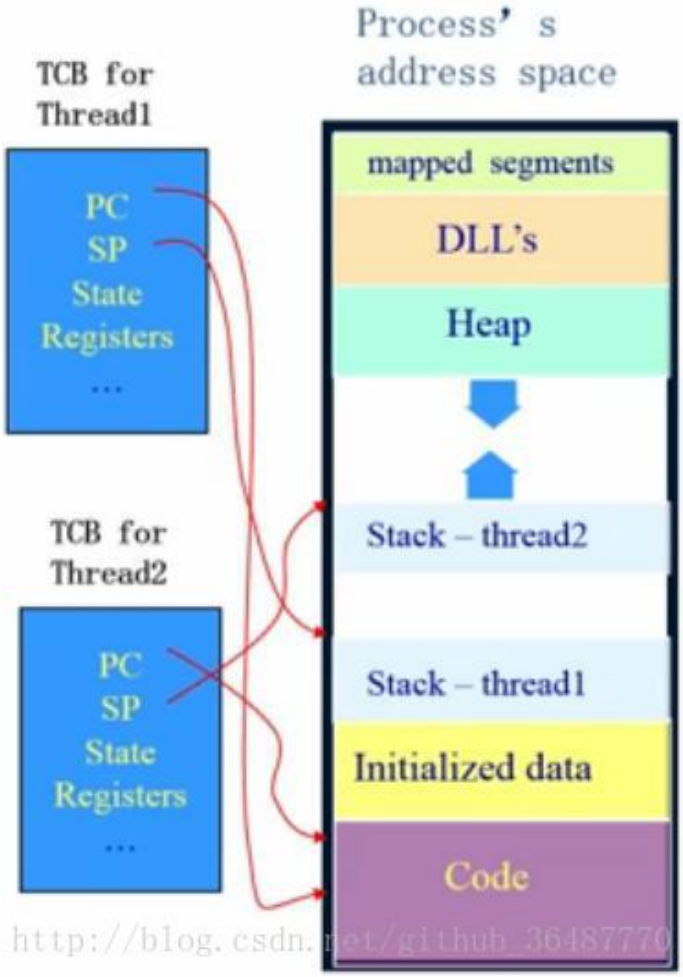
- 引入线程概念的原因：
  - 线程的创建的时空开销比进程小；
  - 线程的撤销的时空开销比进程小；
  - 同进程内线程切换的时空开销比进程小；
  - 由于同进程内线程间共享内存和文件资源，可直接进行不通过内核的通信，效率更高；
- 引入线程减小了系统的基本工作单位粒度，其目的在于：
  - 实现进程内部的并发执行，提高并行程度
  - 减少处理器切换带来的时空开销
  - 简化进程通信方式

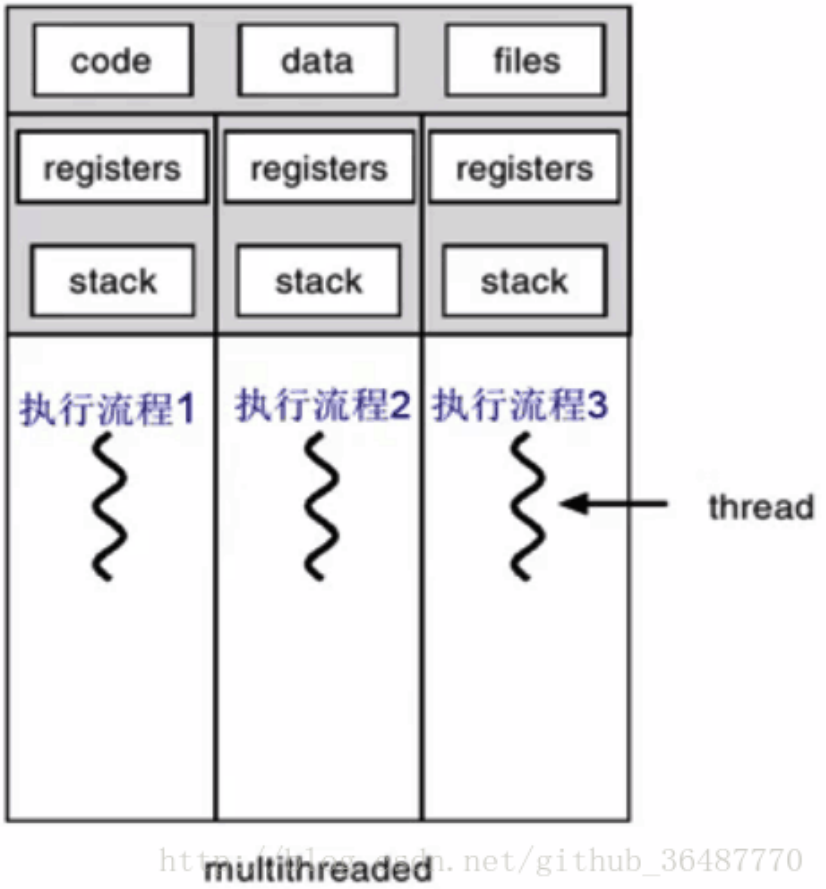
线程

- 进程：
  - 其它资源（除处理器外）分配的基本单位
    - 包括地址空间（代码段，数据段），打开的文件，访问的网络等资源
  - 处理器分配调度的基本单位？？
    - 即代码在资源平台上的一条执行流程
  - 将进程两个基本单位分开考虑，进程=共享资源+线程（引入）
- 线程：
  - 把进程细化成若干个可以独立运行的实体，每一个实体称为一个线程(Thread)
  - 处理器分配调度的基本单位

线程

- 线程所需的资源(共享/独享/关系式)和线程控制块TCB(组成)





http://blog.csdn.net/github_36487770

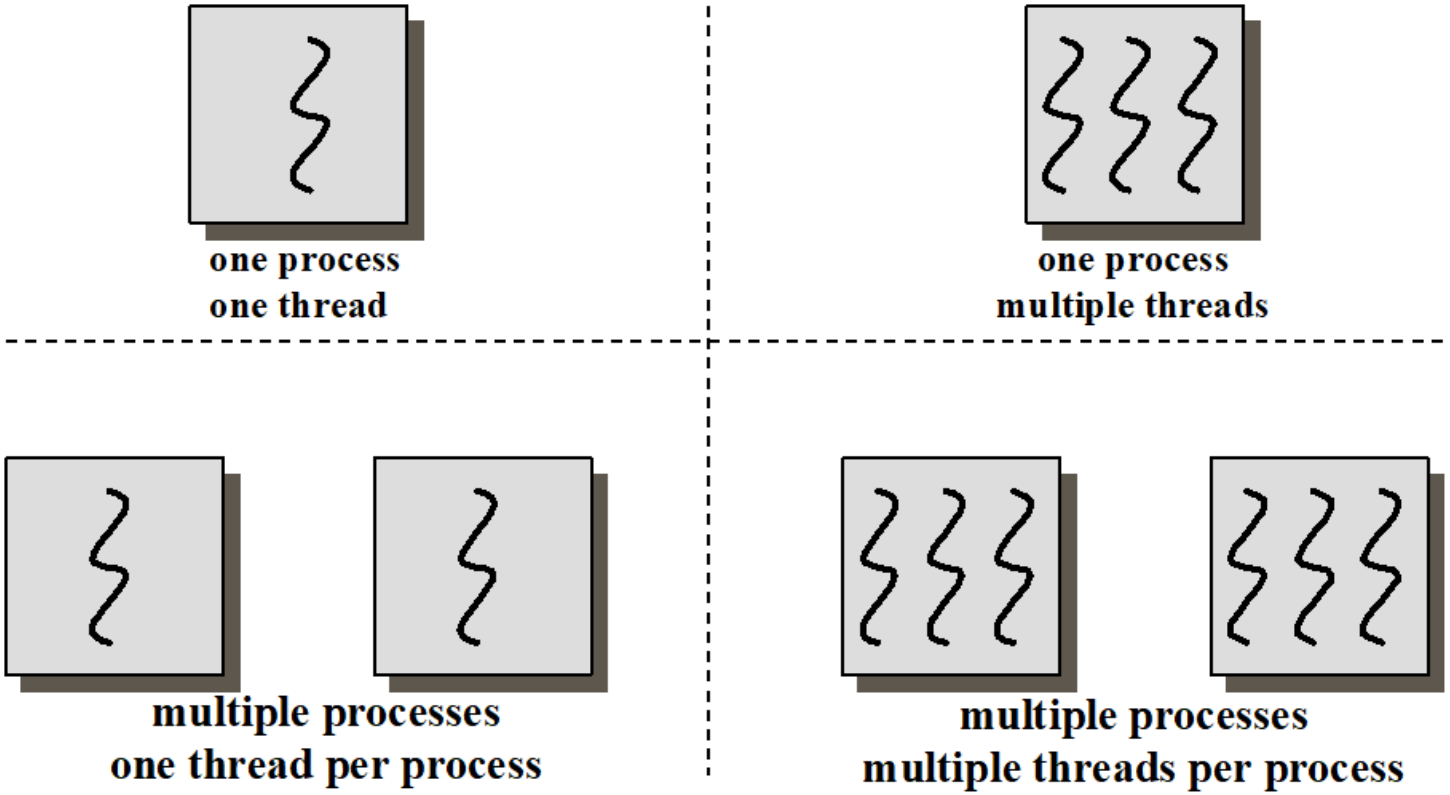
线程

❖ 线程与进程的比较

对比项目	进程	线程
独立拥有的资源	处理器控制权和其他资源	处理器控制权
内存访问	不共享	同进程内共享
切换时的时空开销	大	小
三种基本状态、动态性、并发性 处理同步/互斥	相同	

线程

- 不同的OS对线程的支持是不一样的



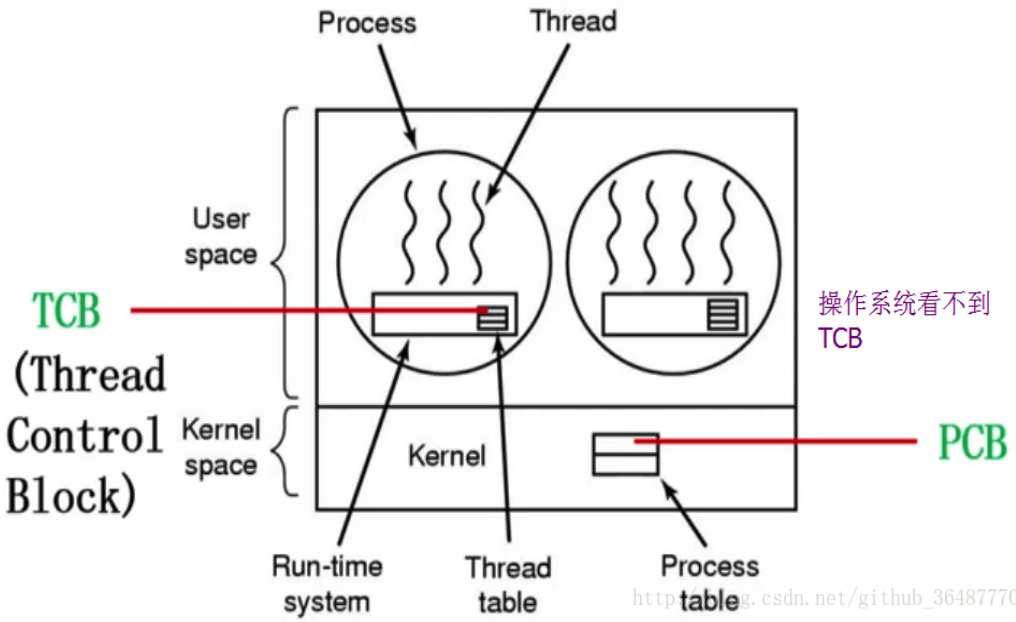
线程

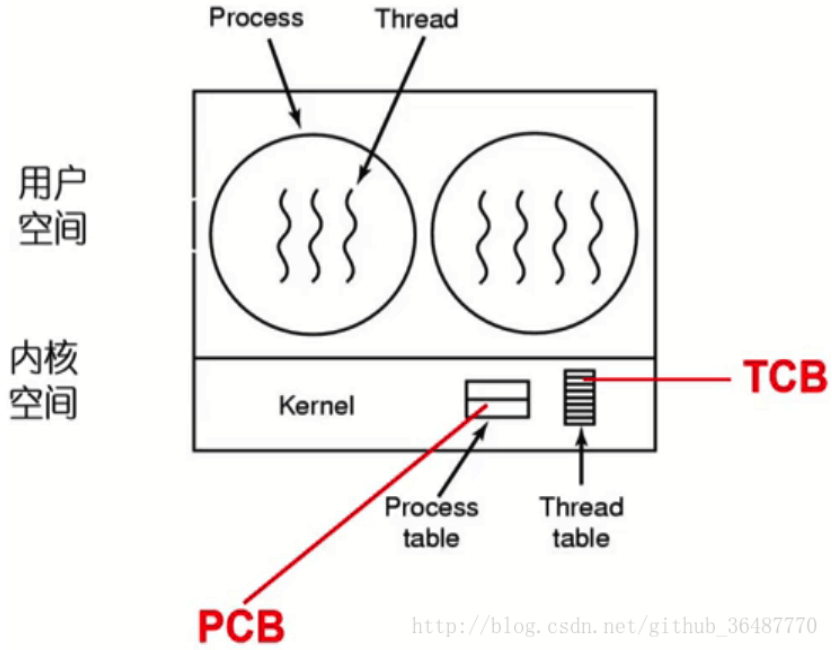
❖ 线程优缺点及应用分析

- ➢ 优点：线程是控制流的管理，一个进程中可以同时存在多个线程，各个线程并发执行，共享地址空间和文件等资源。(便捷)
- ➢ 缺点：安全可靠性不行，一个线程崩溃，会导致这个进程的所有线程都崩溃。(因为共享)
- ➢ 适用性的问题！适用范围：
  - 高性能计算中很强调性能
  - 网络INTERNET一些服务

线程的实现机制

- 对线程实施管理、控制的模块称作线程包（Threads Package）。根据线程包的实现方式，将线程分为：
  - 用户级线程——由用户态的线程包（例如运行时系统）来管控，而内核中不识别此类线程
  - 系统级线程——使用内核提供的线程包来管控





http://blog.csdn.net/github_36487770

线程的实现机制

- 用户级线程
  - 优点：
    - 用户级线程的管控由相应的进程调用用户态的线程包完成，不需要操作系统内核了解用户线程的存在，可用于不支持线程技术的多进程操作系统（每个进程都有私有的线程控制块(TCB)列表，用来跟踪记录记录它的各个线程的状态信息，TCB由用户态的线程包创建终止和管理）
    - 处理器切换由用户态的线程包完成，无需进入内核，节省开销
  - 缺点：
    - 某线程在内核上阻塞时导致全进程阻塞
    - 一个用户线程开始运行后，除非主动交出CPU使用权，否则所有进程中的其他线程都无法运行
    - 时间片分配给进程，多线程中每个线程得到时间片较少

线程的实现机制

- 系统级线程——使用内核提供的线程包来管控
  - 优点：
    - 某线程在内核上阻塞时，同进程的其他线程仍可运行
    - 时间片分配给线程，多线程的进程得到更多CPU时间片
  - 缺点：
    - 系统级线程的管控由OS调用内核中线程包完成，需要支持内核线程的操作系统（OS的调度单位不是进程而是线程，进程主要完成资源的管理，TCB由内核中线程包创建终止管理）
    - 切换由系统调用/调用内核函数完成，需进入内核，系统开销比较大

进程的常用细化方法

- 将进程细化为线程需要根据实际应用的需要，主要的细化方法有：
  - 1. 分派/处理模型（Dispatcher / Worker Model）

    由一个分派线程（Dispatcher）负责接收工作需求，然后将工作任务分派给工作线程（Worker）以完成具体动作。

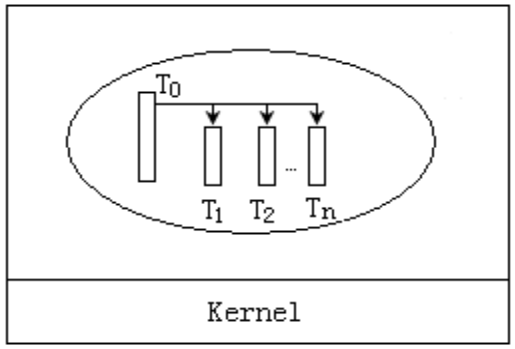


图3-16 分派/处理模型

Dispatcher和Worker之间是主从关系

其中，Dispatcher处于控制地位

进程的常用细化方法

- 2. 队列模型（Team Model）

  将进程细化为具有独立关系的线程，各自完成独立的任务

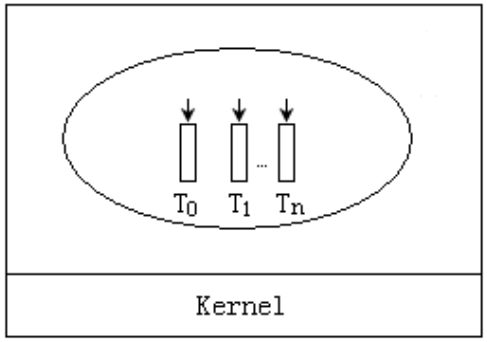


图3-17 队列模型

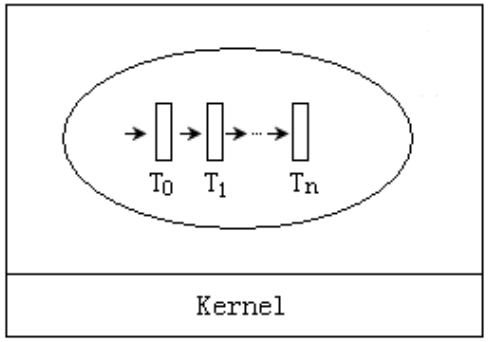


图3-18 管道模型

- 3. 管道模型（Pipeline Model）

  将完整进程中有序的各执行步骤作为线程，依次顺序运行（流水线）

第三章测试题

6. 进程控制的唤醒原语，实现进程的一种状态变化，下列正确的是（　　）。

A. 就绪态→运行态　　B. 就绪态→阻塞态　　C. 阻塞态→运行态　　D. 阻塞态→就绪态

7. 关于线程，下列错误的是（　　）。

A. 线程与进程之间可以并发执行　　B. 线程的采用实现 CPU 管理的更小粒度化

C. 线程可以减少 CPU 切换时的系统开销　　D. 进程细化为线程，是由操作系统自动完成

8. 在信号量机制的 p()内部，调用 p(s)操作的进程进入阻塞状态的条件是(　　)

A. s. value>0　　B. s. value<0　　C. s. value=0　　D. s. value≤0

8. 下列关于进程和线程的叙述中，正确的是（　　）。

A. 不管系统是否支持线程，进程都是资源分配的基本单位

B. 线程是资源分配的基本单位，进程是调度的基本单位

C. 系统级线程和用户级线程的切换都需要内核的支持

D. 同一进程中的各个线程拥有各自不同的地址空间

9. 在信号量机制的 p()内部，调用 p(s)操作的进程进入阻塞状态的条件是(　　)。

A. s. value>0　　B. s. value<0　　C. s. value=0　　D. s. value≤0

10. 在由 9 个生产者，6 个消费者，共享容量为 3 的缓冲器组成的生产者-消费者问题中，互斥使用缓冲器的信号量 mutex 的初值应该为（　　）。

A. 1　　B. 3　　C. 6　　D. 9